

Claude Sandbox

Technical Reference

Operational patterns for building applications inside claude.ai

FORMAT: Verified facts and patterns only. No narrative. Closed units. No open threads.

1. Environment Facts

1.1 Container Baseline

Property	Value
OS	Ubuntu 24.04 LTS (Noble), amd64
PID 1	/process_api — Anthropic binary. Enforces all limits. Do not interact with it.
RAM limit	4GB hard cap, OOM-poled every 100ms by process_api
Network egress	BLOCKED. No outbound connections from container scripts.
Display	No physical display. Xvfb virtual framebuffer on :99 available.
Session persistence	Filesystem resets between sessions. /mnt/user-data/outputs persists within session.
User uploads	Available at /mnt/user-data/uploads (read-only from container)

1.2 Pre-installed Software (do not reinstall)

Category	Available Tools
Runtimes	Python 3.12, Node.js 22, Java, uv
Media	ffmpeg, ImageMagick (import/convert/mogrify), ffprobe
Documents	LibreOffice, Pandoc, LaTeX, wkhtmltopdf, pdftk, tesseract OCR
Display stack	Xvfb, GTK3, gstreamer, PipeWire
Browsers	Playwright + Chromium (headless)
Dev	graphviz, git, curl, build-essential, strace
Python packages	numpy, PIL/Pillow, cv2 (opencv), requests — check pip list before installing
Fonts	Poppins, Lora, DejaVu, Liberation, full Google Fonts collection

1.3 Key File Paths

Path	Purpose
/mnt/user-data/uploads/	User-uploaded files. Read-only.
/mnt/user-data/outputs/	Write final deliverables here to make them presentable to user.

Path	Purpose
/home/claude/	Working directory. Use for scripts, logs, intermediate files.
/dev/shm/	RAM-backed filesystem. Use for inter-process file exchange. Fastest I/O available.
/mnt/skills/	Claude skill documentation. Read-only.
/usr/share/fonts/truetype/google-fonts/	All brand and Google fonts.

2. Process Model

2.1 The Session Reaper — Critical Constraint

CRITICAL Bash tool calls have a timeout (~10-15s on blocking calls).
On timeout: the bash tool's process group is killed. All child processes die with it.
nohup is NOT sufficient — it only handles SIGHUP, not process group termination.
Implication: any process you need to survive between messages **MUST** use the double-fork pattern.

2.2 Double-Fork Daemon — The Only Reliable Launch Pattern

Fully detaches the target process from the bash tool's process group. Survives between messages.

```
# /home/claude/launch.py - run this to start any persistent process
import os, sys

pid = os.fork()
if pid > 0: sys.exit(0)          # Parent exits

os.setsid()                     # Become session leader

pid2 = os.fork()
if pid2 > 0: sys.exit(0)        # First child exits

# Grandchild: fully detached from original process group
sys.stdin = open(os.devnull)
log = open('/home/claude/app.log', 'w')
os.dup2(log.fileno(), 1)
os.dup2(1, 2)

os.execv('/bin/bash', ['bash', '/home/claude/supervisor.sh'])
```

```
# Launch it:
python3 /home/claude/launch.py
# Returns immediately. Grandchild is fully detached.
sleep 2 && cat /home/claude/app.log # Verify startup
```

2.3 Supervisor Pattern — Infrastructure vs Application

Rule: infrastructure (Xvfb, ffmpeg recording) starts ONCE outside the loop. Only the application logic restarts.

```
#!/bin/bash
# /home/claude/supervisor.sh

# — INFRASTRUCTURE (start once, never restart) —
pkill -f Xvfb 2>/dev/null
rm -f /tmp/.X11-unix/X99 /tmp/.X99-lock
sleep 0.3
Xvfb :99 -screen 0 640x480x24 > /dev/null 2>&1 &
until [ -S /tmp/.X11-unix/X99 ]; do sleep 0.3; done
export DISPLAY=:99 SDL_AUDIODRIVER=dummy

# Start your application binary
myapp --config /home/claude/myapp.cfg > /home/claude/myapp.log 2>&1 &
sleep 2

# Persistent video recording (start once, leave running)
ffmpeg -f x11grab -r 6 -s 640x480 -i :99.0 \
    -c:v libx264 -preset ultrafast /home/claude/recording.mkv -y \
    > /dev/null 2>&1 &

# — APPLICATION LOGIC WATCHDOG (restarts on crash) —
while true; do
    echo "[$(date +%H:%M:%S)] navigator start" >> /home/claude/supervisor.log
    # Log memory before each restart - lets you see OOM trend in post-mortem
    free -m | grep Mem >> /home/claude/supervisor.log
    python3 /home/claude/app_logic.py >> /home/claude/app.log 2>&1
    echo "[$(date +%H:%M:%S)] navigator exit $" >> /home/claude/supervisor.log
    sleep 2
done
```

2.4 X11 Heartbeat — Required for Long-Running Display Connections

Without periodic X11 event queue activity, the display server considers the connection dead and terminates it.

```
# Add to any Python process that holds an X11 connection
import threading, subprocess, time

def heartbeat():
    while True:
        subprocess.run(['xdotool', 'key', 'F12'],
                        env={"DISPLAY": ":99"}, capture_output=True)
        time.sleep(1.0) # 1s is sufficient; 3s is risky under load

threading.Thread(target=heartbeat, daemon=True).start()
# daemon=True: thread dies with the process, does not block shutdown
```

3. Headless Display Stack

3.1 Xvfb Setup and Recovery

```
# Standard start sequence (always clean locks first)
pkill -f Xvfb 2>/dev/null
rm -f /tmp/.X11-unix/X99 /tmp/.X99-lock
sleep 0.3
Xvfb :99 -screen 0 640x480x24 > /dev/null 2>&1 &
```

```
# Poll for ready - socket file appears when Xvfb is accepting connections
for i in {1..20}; do
    [ -S /tmp/.X11-unix/X99 ] && echo "ready" && break
    sleep 0.3
done

# Verify display is alive at any point (ffmpeg probe - no xdpinfo needed)
DISPLAY=:99 ffmpeg -f x11grab -i :99 -vframes 1 /tmp/probe.png -y 2>&1 | tail -1
```

3.2 Frame Capture — Tool Selection

Use Case	Tool	Notes
Continuous recording	ffmpeg -f x11grab	Start once as daemon. Output to MKV.
Single frame for analysis	import -window root /tmp/f.png	ImageMagick. Pre-installed. Fast.
Raw pixel buffer	xwd -root -display :99	Fastest. Uncompressed XWD format.
Base64 frame for artifact	import -window root png:- base64	Pipe directly to base64

CRITICAL

Always use MKV for video output. Never MP4.
MP4 writes MOOV atom at file end. If ffmpeg is killed mid-stream, the file is permanently unrecoverable.
MKV writes headers inline. Partial MKV always recoverable: ffmpeg -i partial.mkv -c copy output.mp4

3.3 Input Injection

REQUIRED

Always target --window WINDOW_ID. Without it, focus-stealing by menus/dialogs silently swallows input.
Get window ID: DISPLAY=:99 xdotool search --name "appname" | head -1

```
# Find window ID
WIN=$(DISPLAY=:99 xdotool search --name "myapp" | head -1)

# Send keys to specific window
xdotool key --window $WIN Return
xdotool key --window $WIN Left Left Left

# Key duration is critical for games/interactive apps
# 35ms = sub-threshold (barely registers in most apps)
# 75ms = turns/rotations (prevents over-rotation)
# 300ms = forward movement (actually traverses space)
# 100ms = action keys (attack, interact)
xdotool key --window $WIN --delay 300 w # forward
xdotool key --window $WIN --delay 75 Left # turn

# Mouse
xdotool mousemove --window $WIN 320 240
xdotool click --window $WIN 1
```

INPUT JITTER

xdotool timing depends on container CPU load. A 300ms keypress may take 400ms+ under load.

Symptom: inputs registering correctly in isolation but inconsistently under sustained operation.
Diagnostic: before checking code logic, check CPU/memory first — `free -h` and `top -bn1 | head -5`.
If container is CPU-starved, reduce capture rate (lower Hz) before tuning key timings.

4. Application Architecture

4.1 Capture-Analyse-Act — Core Pattern

Decouple all three phases. They run at different rates. Never couple action rate to frame grab rate.

Phase	Rate	Tool	Output
Capture	2 Hz	<code>import -window root</code>	PNG to <code>/dev/shm/</code>
Analyse	1 Hz	Python/numpy pixel ops	Actions to queue
Act	20+ Hz	<code>xdotool</code>	Keypresses to app window

```
import threading, queue, time, subprocess
from PIL import Image

action_queue = queue.Queue(maxsize=20)

def capture_loop():
    while True:
        subprocess.run(['import', '-window', 'root', '/dev/shm/frame.png'],
                       env={"DISPLAY": ":99"}, capture_output=True)
        time.sleep(0.5)

def analyse_loop():
    while True:
        try:
            img = Image.open('/dev/shm/frame.png')
            for action in decide(img):
                action_queue.put_nowait(action)
        except: pass
        time.sleep(1.0)

def act_loop():
    while True:
        try:
            action = action_queue.get_nowait()
            send_input(action)
        except queue.Empty:
            send_input('default_action') # fallback keeps app responsive
        time.sleep(0.05)

for fn in [capture_loop, analyse_loop, act_loop]:
    threading.Thread(target=fn, daemon=True).start()
```

4.2 /dev/shm — Inter-Process Communication

RAM-backed filesystem. Use for all inter-process file exchange. Eliminates disk I/O race conditions.

```
# Standard layout
mkdir -p /dev/shm/{frames,state,actions}

# Write frame
DISPLAY=:99 import -window root /dev/shm/frames/latest.png

# Share state between processes (Python)
import json
with open('/dev/shm/state/app.json', 'w') as f: json.dump(state, f)

# Read state (bash)
cat /dev/shm/state/app.json | python3 -c "import sys,json; d=json.load(sys.stdin);
print(d['key'])"
```

4.3 Pixel Analysis — Lightweight Decision Making

Pixel statistics are 1000x faster than ML inference and sufficient for most real-time decisions.

```
import numpy as np
from PIL import Image

def analyse(path):
    arr = np.array(Image.open(path).convert('RGB'))
    h, w, _ = arr.shape

    # Detect UI overlays (menus, dialogs) by colour signature
    # Sample top 75% only - excludes bottom-anchored HUD elements
    top = arr[:int(h*0.75)]
    red_pixels = np.sum((top[:, :, 0] > 180) & (top[:, :, 1] < 80) & (top[:, :, 2] < 80))

    # Detect wall/obstacle ahead by centre column variance
    # Low variance = uniform surface (wall). High = open space.
    center = arr[:, w//3:2*w//3, :]
    center_var = float(np.std(center))

    # Detect navigable direction by left/right brightness
    left_open = float(np.mean(arr[:, :w//3, :]))
    right_open = float(np.mean(arr[:, 2*w//3:, :]))

    # Detect motion (stuck detection)
    # Compare current frame to previous via stored file

    return {
        "ui_overlay": red_pixels > 5000,
        "wall_ahead": center_var < 15.0,
        "go_right": right_open > left_open,
    }
```

4.4 Audio Configuration

PATTERN

Set `SDL_AUDIODRIVER=dummy` as env var BEFORE launching any SDL application. In config files: `snd_sfxdevice 0` (works), `snd_sfxdevice -1` (causes `I_Init` hang — never use). Config files override CLI flags in many apps. If `-nosound` flag is ignored, the app found a config. Delete it.

```
# Required environment for any SDL app in the sandbox
```

```
export SDL_AUDIODRIVER=dummy
export DISPLAY=:99

# If config file disables overrides, write a clean one first
cat > ~/.config/myapp/myapp.cfg << 'EOF'
snd_sfxdevice 0
snd_musicdevice 0
EOF

# Then launch
myapp --iwad /path/to/data.wad
```

5. Software Installation

5.1 Package Upload Pipeline

Network egress is blocked. All packages must be uploaded by the user as file attachments.

1. Identify exact package: Ubuntu 24.04 Noble, amd64. Use packages.ubuntu.com.
2. Download .deb on user's machine: `apt-get download packagename`
3. User uploads .deb as attachment to conversation
4. Install: `dpkg -i /mnt/user-data/uploads/package.deb`
5. If dependency errors: read output, repeat from step 1 for each missing dep
6. Record every successfully installed package — sessions reset

```
# Install uploaded .deb
dpkg -i /mnt/user-data/uploads/package_version_amd64.deb

# Typical error output requiring follow-up:
# dpkg: dependency problems prevent configuration:
#  package depends on libfoo (>= 2.0.0)
# → upload and install libfoo_2.x.x_amd64.deb, then retry

# Verify installation
dpkg -l | grep packagename
```

5.2 Python Packages

```
# Check what's already available first
python3 -c "import numpy, cv2, PIL; print('pre-installed')"
```

```
# Install with --break-system-packages (required in this environment)
pip install packagename --break-system-packages

# Install from uploaded wheel (when network blocked)
pip install /mnt/user-data/uploads/package-version-cp312-linux_x86_64.whl \
  --break-system-packages
```

6. Accessibility Tool Patterns

6.1 Text-to-Speech Pipeline

```
# espeak-ng is available (basic quality)
espeak-ng -v en-us "text here" -w /dev/shm/speech.wav

# Full pipeline with file output
def tts(text, output_path):
    with open('/dev/shm/tts_in.txt', 'w') as f: f.write(text)
    subprocess.run(['espeak-ng', '-v', 'en-us',
                    '-f', '/dev/shm/tts_in.txt',
                    '-w', '/dev/shm/tts_out.wav'], check=True)
    import shutil
    shutil.copy('/dev/shm/tts_out.wav', output_path)
    # present_files([output_path]) to give to user
```

6.2 Screen Region Magnifier

```
from PIL import Image, ImageEnhance
import subprocess

def magnify(x, y, w, h, scale=2.0, contrast=1.5, output='/tmp/mag.png'):
    subprocess.run(['import', '-window', 'root', '/dev/shm/full.png'],
                    env={"DISPLAY": ":99"}, capture_output=True)
    img = Image.open('/dev/shm/full.png').crop((x, y, x+w, y+h))
    img = img.resize((int(w*scale), int(h*scale)), Image.LANCZOS)
    img = ImageEnhance.Contrast(img).enhance(contrast)
    img.save(output)
    return output
```

6.3 OCR Pipeline

```
import subprocess
from PIL import Image

def ocr(x=0, y=0, w=None, h=None):
    subprocess.run(['import', '-window', 'root', '/dev/shm/ocr_raw.png'],
                    env={"DISPLAY": ":99"}, capture_output=True)
    img = Image.open('/dev/shm/ocr_raw.png')
    if w and h: img = img.crop((x, y, x+w, y+h))
    # Preprocess: greyscale + threshold improves accuracy significantly
    img = img.convert('L').point(lambda p: 255 if p > 128 else 0)
    img.save('/dev/shm/ocr_proc.png')
    result = subprocess.run(['tesseract', '/dev/shm/ocr_proc.png', 'stdout'],
                             capture_output=True, text=True)
    return result.stdout.strip()
```

7. Troubleshooting Reference

7.1 Failure Pattern Matrix

Symptom	Root Cause	Fix
Process dies silently between messages	Bash tool timeout killed process group	Double-fork pattern. See §2.2.
App hangs at l_Init / startup indefinitely	SDL audio init blocking	export SDL_AUDIODRIVER=dummy before launch

Symptom	Root Cause	Fix
App hangs despite SDL_AUDIODRIVER=dummy	Config file has snd_sfxdevice -1	Find and delete config file. -1 = hang, 0 = works.
Xvfb dies during operation	X11 event queue starved	Add 1s heartbeat thread. See §2.4.
ffmpeg can't connect to display	Xvfb died, stale lock file	Full cleanup sequence: §7.2
Input keys have no effect	Focus stolen by menu/dialog	Use xdotool --window WINDOW_ID
Video file 0 bytes or unplayable	ffmpeg killed before writing MOOV	Use MKV. Recover: ffmpeg -i partial.mkv -c copy out.mp4
dpkg: dependency problems	Missing .deb dependency	Read error, upload listed dependency, retry
Script works foreground, fails background	Env vars not inherited	Use export. Set env vars before execv/subprocess.
Everything slows then dies	Subprocess storm (too many forks/sec)	Replace per-frame ffmpeg with one persistent daemon
Segfault in X11/ctypes call	NULL display pointer	Verify DISPLAY=:99 exported; verify Xvfb alive first
UI -nosound flag ignored	App found config file overriding flag	Delete config file or overwrite with correct values
Pixel/colour detector fires during deterministic route, aborting it	Spawn room geometry triggers threshold (red=9000+ from wall textures, not a menu)	Never run UI detection inside a deterministic route. Route runs after confirmed game start — no menus exist. Detection belongs in brain thread only.

7.2 Xvfb Full Recovery Sequence

```
# Run before EVERY Xvfb start (not just on failure)
pkill -f Xvfb 2>/dev/null
pkill -f myapp 2>/dev/null
sleep 0.5
rm -f /tmp/.X11-unix/X99
rm -f /tmp/.X99-lock

# Fresh start
Xvfb :99 -screen 0 640x480x24 > /dev/null 2>&1 &

# Poll for readiness via socket file — no xdpyinfo needed
for i in {1..20}; do
    [ -S /tmp/.X11-unix/X99 ] && break
    sleep 0.3
done
echo "Xvfb ready"
```

7.3 Process Diagnostic Sequence

```
# Is it running?
pgrep -la myapp

# What happened?
tail -50 /home/claude/app.log

# OOM kill?
dmesg | grep -i "out of memory" | tail -5
```

```
# State while running (S=sleeping/blocked, R=running/spinning, Z=zombie)
cat /proc/$(pgrep myapp)/status | grep -E "State|VmRSS"

# Blocking syscall (for hangs)
strace -p $(pgrep myapp) -e trace=read,write,futex 2>&1 | head -20

# What file descriptors does it have open?
ls -la /proc/$(pgrep myapp)/fd 2>/dev/null
```

7.4 Blackhole State Detection

Blackhole: process existence and log existence are decoupled. pgrep returns a PID but logs are stale or absent — or pgrep returns nothing and no error was ever written. Standard log checks fail because the output was silently swallowed before it could be written.

TRAP: os.dup2 swallows supervisor output

If launch.py pre-redirects stdout via `os.dup2(log.fileno(), 1)`, and supervisor.sh then does `exec >> logfile`, the exec redirect goes nowhere — fd1 is already bound. The file from `os.dup2` gets the output; the exec file stays empty. Fix: do NOT redirect stdout in launch.py. Let supervisor.sh manage its own log with `exec >> logfile`. Symptom: "grandchild alive" line never appears despite supervisor process existing.

```
# launch.py - stdin only, no stdout redirect
sys.stdin = open(os.devnull)
# NO: log = open(...); os.dup2(log.fileno(), 1)
os.execv('/bin/bash', ['bash', '/home/claude/supervisor.sh'])

# supervisor.sh - manages its own log
exec >> /home/claude/supervisor.log 2>&1 # All output goes here
set -x # Trace every command with timestamp
echo "[$(date +%H:%M:%S)] grandchild alive PID $$" # LINE 1
# If this line is absent: double-fork failed before exec
# set -x output after this line shows exactly which command failed and why
```

DIAGNOSTIC TOOL: set -x

Add `set -x` immediately after `exec >> logfile` in supervisor.sh. Every subsequent command is logged with a + prefix and timestamp. Disable for the watchdog loop (`set +x`) — otherwise it generates noise on every iteration. This is the only reliable way to see which line a silent crash occurred on.

Single-line stack verify — run any time state is uncertain:

```
pgrep -la Xvfb; pgrep -la myapp; pgrep -la ffmpeg; \
ls -lt /home/claude/*.log 2>/dev/null | head -4; \
echo "---"; tail -5 /home/claude/supervisor.log 2>/dev/null; \
tail -3 /home/claude/app.log 2>/dev/null

# Interpret:
# PID present + log recent + has content → healthy
# PID present + log timestamp old (>2min) → running but silent (suspicious)
# No PID + supervisor.log has "grandchild alive" → launched, died after first write
# No PID + no log OR missing "grandchild alive" → double-fork failed / os.dup2 trap
```

```
# supervisor.log empty despite process existing      → os.dup2 redirect conflict — see trap
above
```

NOTE Zombie PID (Z state) looks identical to healthy PID to pgrep. If PID exists but logs are stale:
cat /proc/PID/status | grep State
Z=zombie, S=sleeping/blocked, R=running, D=uninterruptible I/O wait.
D state for >30s = effectively hung. kill -9 is the only option.

8. Quick Reference

8.1 Stack Verify Script — Run First, Always

Single command. Paste and run to get complete stack state. Interpret output against the table below.

```
# VERIFY_STACK — paste this any time state is uncertain
echo "=== PROCESSES ===" && \
  pgrep -laE "Xvfb|ffmpeg|supervisor|myapp|python" 2>/dev/null || echo "none"; \
echo "=== DISPLAY ===" && \
  ([ -S /tmp/.X11-unix/X99 ] && echo "ONLINE :99") || echo "OFFLINE"; \
echo "=== MEMORY ===" && free -h | grep Mem; \
echo "=== LOG RECENCY ===" && ls -lt /home/claude/*.log 2>/dev/null | head -5; \
echo "=== LAST APP LOG ===" && tail -5 /home/claude/app.log 2>/dev/null || echo "no log"; \
echo "=== LAST SUPERVISOR ===" && tail -5 /home/claude/supervisor.log 2>/dev/null || echo "no log"
```

Output pattern	State	Action
PIDs present + display ONLINE + log recent + log has content	Healthy	None
PIDs present + display ONLINE + log timestamp old (>2min)	Running but silent	Check process state: cat /proc/PID/status grep State
No PIDs + log has "grandchild alive" line	Launched, died after start	Read log for next line — that is the failure point
No PIDs + no log OR log missing "grandchild alive"	Double-fork failed entirely	Re-run launch.py. Check for stale X11 locks first.
Display OFFLINE + PIDs present	Xvfb died under running process	Full recovery sequence §7.2. Restart everything.
Memory used >3.5GB	OOM risk	Kill non-essential processes. Reduce capture rate.

8.2 Launch Checklist

- Clean X11 locks and kill stale processes
- Start Xvfb :99, poll for readiness (not sleep)
- export DISPLAY=:99 SDL_AUDIODRIVER=dummy
- Double-fork launch.py → supervisor.sh
- supervisor.sh line 1: write "grandchild alive" to log

12. Supervisor starts: application binary, ffmpeg MKV recording, watchdog loop with free -m logging
13. Application logic line 1: write PID to log. Line 2: start heartbeat thread.
14. Run VERIFY_STACK. Confirm all green before proceeding.

8.3 Minimal Complete Template

```
# — launch.py (double-fork entry point) —
import os, sys
if os.fork() > 0: sys.exit(0)
os.setsid()
if os.fork() > 0: sys.exit(0)
# Redirect stdin only. Do NOT pre-redirect stdout/stderr here.
# If the supervisor script does exec >> logfile, os.dup2 will
# silently swallow all output — the script manages its own log.
sys.stdin = open(os.devnull)
os.execv('/bin/bash', ['bash', '/home/claude/supervisor.sh'])
```

```
# — supervisor.sh —
#!/bin/bash
# Supervisor manages its own log — do NOT pre-redirect in launch.py
# set -x traces every command with timestamp to the log
exec >> /home/claude/supervisor.log 2>&1
set -x
echo "[$(date +%H:%M:%S)] grandchild alive PID $$" # LINE 1 — if absent, double-fork failed
pkill -f Xvfb 2>/dev/null; rm -f /tmp/.X11-unix/X99 /tmp/.X99-lock; sleep 0.3
Xvfb :99 -screen 0 640x480x24 > /dev/null 2>&1 &
until [ -S /tmp/.X11-unix/X99 ]; do sleep 0.3; done
export DISPLAY=:99 SDL_AUDIODRIVER=dummy
myapp --config /home/claude/myapp.cfg > /home/claude/myapp.log 2>&1 &
sleep 2
ffmpeg -f x11grab -r 6 -s 640x480 -i :99.0 -c:v libx264 \
/home/claude/recording.mkv -y > /dev/null 2>&1 &
set +x # Disable trace for watchdog loop — too verbose
while true; do
    echo "[$(date +%H:%M:%S)] restart"
    free -m | grep Mem
    python3 /home/claude/app_logic.py >> /home/claude/app.log 2>&1
    echo "[$(date +%H:%M:%S)] exit $?"
    sleep 2
done
```

```
# — app_logic.py (minimal structure) —
import os, threading, queue, time, subprocess
from PIL import Image
import numpy as np

# LINE 1: synchronous log write before anything else
open('/home/claude/app.log', 'a').write(f"[app_logic] started PID {os.getpid()}\n")

# Heartbeat — start immediately
def heartbeat():
    while True:
        subprocess.run(['xdotool', 'key', 'F12'], env={"DISPLAY": ":99"}, capture_output=True)
        time.sleep(1.0)
threading.Thread(target=heartbeat, daemon=True).start()

WIN = None
def get_win():
    global WIN
    r = subprocess.run(['xdotool', 'search', '--name', 'myapp'],
        capture_output=True, text=True, env={"DISPLAY": ":99"})
    WIN = r.stdout.strip().split('\n')[0] if r.stdout.strip() else None
```

```
def send(key, delay_ms=75):
    if WIN:
        subprocess.run(['xdotool', 'key', '--window', WIN, '--delay', str(delay_ms), key],
                        env={"DISPLAY": ":99"}, capture_output=True)

def grab():
    subprocess.run(['import', '-window', 'root', '/dev/shm/frame.png'],
                    env={"DISPLAY": ":99"}, capture_output=True)
    return np.array(Image.open('/dev/shm/frame.png').convert('RGB'))

get_win()
while True:
    arr = grab()
    # analyse arr, call send() based on result
    time.sleep(0.5)
```

8.4 Key Commands Cheatsheet

```
# Status
ps aux | grep -E "Xvfb|ffmpeg|python" | grep -v grep
free -h

# Display check (socket file - no xdpinfo)
[ -S /tmp/.X11-unix/X99 ] && echo "Xvfb up" || echo "Xvfb down"
DISPLAY=:99 import -window root /tmp/check.png && echo "display responsive"

# Window targeting
DISPLAY=:99 xdotool search --name ".*" 2>/dev/null

# Kill and clean
pkill -f Xvfb; pkill -f ffmpeg; pkill -f navigator
rm -f /tmp/.X11-unix/X99 /tmp/.X99-lock

# MKV recovery
ffmpeg -i partial.mkv -c copy recovered.mp4

# Package management
dpkg -i /mnt/user-data/uploads/package.deb
pip install name --break-system-packages
dpkg -l | grep name

# Logs
tail -f /home/claude/app.log
tail -f /home/claude/supervisor.log
```

9. Case Study: Production TTS System

Built directly from sandbox patterns. Converts text to natural-sounding speech with no artifacts. Accessibility use case: aphasia + vision issues.

RESULT

Final output: 48kHz 16-bit mono PCM, RHVoice Q5, no stuttering, no clicks, consistent playback.
Generation speed: approximately real-time (1 minute text $\approx$ 1 minute generation).
Memory: $\sim$ 200MB peak. File size: $\sim$ 10MB per minute.

9.1 Diagnosis Journey — What The Problem Was Not

Ten attempts eliminated wrong hypotheses before finding the real cause. Each elimination is documented because the same wrong turns are likely for any audio pipeline.

Hypothesis	Test	Result
TTS engine is the problem	Switched espeak → RHVoice	Same stuttering. Engine eliminated.
I/O speed is the problem	Moved everything to /dev/shm	Still stuttering. I/O eliminated.
Buffering is the problem	stdbuf -o0, large ffmpeg buffers	Still stuttering. Buffering eliminated.
Subprocess storm is the problem	Persistent daemon pattern	Still stuttering. Spawn overhead eliminated.
Pipe streaming is the problem	RHVoice → pipe → ffmpeg	Still stuttering. Pipes eliminated.
Multi-core pinning (Attempt 9)	taskset -c 0 synthesis, taskset -c 1 encoding	Audio smooth — but playing at 400x speed. Breakthrough: stuttering fixed, new problem found.
Sample rate mismatch	Corrected ffmpeg sample rate handling	Normal speed — but playback sounded different each time. New clue.
FFmpeg concatenation discontinuities (ACTUAL CAUSE)	Crossfade chunks with 50ms triangular overlap	Clean audio. Consistent playback. Problem solved.

KEY INSIGHT

Stable MD5 does not mean good audio. A file can be perfectly stable but contain packaging defects.

Playback sounding different each time is not file corruption — players handle amplitude discontinuities differently.

When concatenating audio chunks without crossfading, the speaker cone must jump between amplitudes in one sample (1/48000s). This creates clicks, waveform spikes, and player-dependent behavior.

9.2 Root Cause: FFmpeg Discontinuity Artifacts

Concatenating audio chunks without crossfading creates abrupt amplitude jumps at boundaries:

```
# The problem — abrupt boundary:
# Chunk 1 ends:  amplitude = -0.2
# Chunk 2 starts: amplitude = +0.1
# Speaker cone must jump in one sample = click/pop/spike

# Visible as: vertical spikes in waveform, striations in spectrogram
# Audible as: clicks, pops, inconsistent playback between players

# Diagnose discontinuities:
ffprobe -v error output.wav  # Look for sample rate / format warnings

# Check MD5 stability (rules out filesystem instability):
md5sum output.wav && sleep 3 && md5sum output.wav
# Same MD5 but bad audio = packaging problem, not synthesis

# Test raw PCM to isolate synthesis vs packaging:
ffmpeg -i output.wav -f s16le -ar 48000 raw.pcm
ffplay -f s16le -ar 48000 -ch_layout mono raw.pcm
# Raw plays smooth = WAV packaging is the problem
```

```
# Raw stutters = synthesis is the problem
```

9.3 The Solution: Multi-Core + Crossfade

```
#!/bin/bash
# rhvoice_tts_final.sh - production TTS pipeline

# 1. Preload voice model to RAM (prevents MMF thrashing)
cat /usr/share/RHVoice/voices/evgeniy-eng/24000/voice.data > /dev/null

# 2. All temp files on RAM disk
mkdir -p /dev/shm/tts

# 3. Synthesise each clause - pinned to Core 0
# (smart_chunker splits on .!?!; boundaries)
i=0
while IFS= read -r chunk; do
    echo "$chunk" > /dev/shm/tts/chunk_${i}.txt
    taskset -c 0 RHVoice-test -p English -R 48000 -q 5 \
        -i /dev/shm/tts/chunk_${i}.txt \
        -o /dev/shm/tts/chunk_${i}.wav
    i=$((i+1))
done < chunks.txt

# 4. Crossfade all chunks - pinned to Core 1
# acrossfade: d=50ms overlap, tri curves = zero discontinuity
taskset -c 1 ffmpeg \
    $(for f in /dev/shm/tts/chunk_*.wav; do echo "-i $f"; done) \
    -filter_complex "acrossfade=d=0.05:c1=tri:c2=tri" \
    -ar 48000 -acodec pcm_s16le output.wav

# 5. Atomic sync to prevent partial writes
sync
```

9.4 Python Implementation

```
import re, subprocess, os

def smart_chunker(text):
    """Split on clause boundaries, preserve prosody."""
    chunks = re.split(r'(?<=[.!?!;])\s+', text)
    return [c.strip() for c in chunks if c.strip()]

def synthesise_chunk(text, index, rate=48000, quality=5):
    chunk_txt = f'/dev/shm/tts/chunk_{index}.txt'
    chunk_wav = f'/dev/shm/tts/chunk_{index}.wav'
    with open(chunk_txt, 'w') as f:
        f.write(text)
        f.flush()
        os.fsync(f.fileno()) # Atomic write before synthesis reads it
    subprocess.run([
        'taskset', '-c', '0',
        'RHVoice-test', '-p', 'English',
        '-R', str(rate), '-q', str(quality),
        '-i', chunk_txt, '-o', chunk_wav
    ], check=True)
    return chunk_wav

def crossfade_chunks(wav_files, output, overlap=0.05):
    """Crossfade all chunks. overlap in seconds (0.05 = 50ms)."""
    if len(wav_files) == 1:
```

```
import shutil; shutil.copy(wav_files[0], output); return
inputs = []
for f in wav_files: inputs += ['-i', f]
subprocess.run([
    'taskset', '-c', '1', 'ffmpeg',
    *inputs,
    '-filter_complex', f'acrossfade=d={overlap}:c1=tri:c2=tri',
    '-ar', '48000', '-acodec', 'pcm_s16le', output, '-y'
], check=True)

def tts(text, output_path):
    os.makedirs('/dev/shm/tts', exist_ok=True)
    # Preload voice model
    subprocess.run(['cat', '/usr/share/RHVoice/voices/evgeniy-eng/24000/voice.data'],
        stdout=subprocess.DEVNULL)
    chunks = smart_chunker(text)
    wav_files = [synthesise_chunk(c, i) for i, c in enumerate(chunks)]
    crossfade_chunks(wav_files, output_path)
```

9.5 Dependencies

All installed via .deb upload. Install in this order to satisfy dependency chain:

Package	Notes
libportaudio2	Audio I/O library
libsonic0 + sonic	Pitch/speed processing
libespeak1 + espeak-data + espeak	Fallback TTS (lower quality)
libao4	Audio output abstraction
librhvoice-core7	RHVoice core library
librhvoice-audio2	RHVoice audio layer
rhvoice	RHVoice engine
rhvoice-english	34MB voice file — split into 2x17MB parts for upload, reassemble with: cat part_01.bin part_02.bin > rhvoice-english.deb

```
# Install sequence
for pkg in libportaudio2 libsonic0 sonic libespeak1 espeak-data espeak libao4 \
    librhvoice-core7 librhvoice-audio2 rhvoice; do
    dpkg -i --force-depends /mnt/user-data/uploads/${pkg}*.deb
done

# Reassemble and install split voice file
cat /mnt/user-data/uploads/part_01.bin /mnt/user-data/uploads/part_02.bin > /tmp/rhvoice-english.deb
dpkg -i --force-depends /tmp/rhvoice-english.deb

# Verify
echo "test" | RHVoice-test -p English -R 48000 -o /tmp/test.wav && echo "RHVoice OK"
```

9.6 Debugging Reference

Symptom	Diagnostic	Fix
Audio stutters	md5sum output.wav && sleep 3 && md5sum output.wav — if changes: filesystem issue. If stable: synthesis or packaging.	Test raw PCM to isolate. See §9.2.
Plays at wrong speed	ffprobe output.wav grep Hz — check rate match	Ensure synthesis rate (-R) matches ffmpeg encoding rate. No resampling.
Sounds different each time	Open in audio editor, look for waveform spikes at chunk boundaries	Add/increase crossfade. Minimum 10ms. 50ms recommended.
Synthesis very slow	Check CPU: top -bn1 head -10	Lower quality (-q 3), lower rate (-R 24000), or split on paragraphs not sentences.
Memory error / OOM	free -h during generation	Process chunks incrementally. Lower sample rate (24kHz = half memory of 48kHz).

9.7 Patterns From The Doom Sessions Applied

Doom Pattern	TTS Application
/dev/shm for IPC	All chunk files (text + WAV) in /dev/shm/tts/ — eliminates disk I/O bottleneck
No subprocess storm	All chunks processed in single script. One persistent synthesis call per chunk, not one process per word.
Preload large files	cat voice.data > /dev/null forces 34MB model into page cache before synthesis begins
Explicit sync points	os.fsync() after writing chunk text — prevents synthesis reading incomplete file
Streaming with large buffers	-bufsize 4M -maxrate 2M in ffmpeg — prevents kernel flush spikes
Multi-core decoupling	taskset -c 0 synthesis, taskset -c 1 encoding — eliminates cache thrashing between processes